

MILESTONE 3 ASSIGNMENT 2

Use of Test Data-Driven Automation in Python Behave Framework

Name:	Roshni Ghosh Tagore
Enrollment No:	12023002028175
Course:	Python for Automation
Academic Year:	2023–2027
Department:	Computer Science and Engineering (Artificial Intelligence & Machine Learning)
Campus:	IEM Newtown

Department of Computer Science and Engineering (AI & ML)
Institute of Engineering & Management (IEM), Newtown Campus

1. PROBLEM STATEMENT

In automated software testing, running test suites against single input datasets often leaves edge cases and parameter variations unverified. Re-writing individual test scripts for every dataset leads to massive code duplication and maintenance overhead. This assignment addresses these issues by adapting the **Python Behave BDD Framework** to perform **Data-Driven Automation (for APIs)** using Gherkin Scenario Outline tables and parameterized Python step implementations.

2. OBJECTIVE

The main objectives of this assignment are:

- Adapt the Python BDD Behave framework to handle automated REST API testing scenarios.
- Implement a data-driven testing approach using Gherkin Scenario Outline and Examples tables.
- Execute multiple iterations of identical test scenarios using parameterized input variables.
- Perform automated HTTP POST requests using the Python requests module and validate JSON payloads and response status codes.
- Ensure clean, modular, and reusable step definition methods for API interaction.

3. TOOLS, SOFTWARE, AND CONCEPTS USED

Tool / Concept	Description & Purpose
Python 3.x	Primary programming language providing execution logic and API assertion rules.
Behave Framework	BDD engine supporting parameterized steps, dynamic parsing, and Scenario Outline execution.
Requests Library	Python library utilized to handle HTTP GET/POST calls, request headers, and JSON body payloads.
PyCharm Community	Development environment used for managing framework components, feature files, and step packages.
Gherkin Data Tables	Syntax structure enabling multi-column dataset iterations (Examples :) passed into automated steps.

Data-Driven Approach: Using Scenario Outline allows a single test logic block to automatically run multiple times across various data combinations without code duplication.

4. SOURCE CODE & IMPLEMENTATION STEPS

Step 1: Data-Driven Feature File (features/api_data_driven.feature)

features/api_data_driven.feature

Gherkin Spec

```
Feature: API Data-Driven Testing Scenario Outline: Create user via API with multiple dat
  " and job "

  " Then the response status code should be 201 And the response body should contain nam

  " and job "

  " Examples: | name | job | | Alice | Developer | | Bob | Manager | | Charlie | Teste
```

Step 2: Parameterized API Step Definitions (features/steps/api_steps.py)

features/steps/api_steps.py

Python Logic

```
import requests from behave import given, when, then @given('the API endpoint is "{endpo
```

```
@then('the response body should contain name "{name}" and job "{job}"') def step_impl(context):
```

5. OUTPUT EXPECTED

Feature: API Data-Driven Testing Scenario Outline: Create user via API with multiple datasets

Figure 2.1: Simple Lines of Execution Output for Data-Driven API Scenarios

6. RESULT

Execution Summary: The Behave test runner successfully iterated through all three data rows defined in the Examples table. For each row, an HTTP POST request was dispatched to the endpoint, returning an HTTP 201 Created status and confirming valid dynamic JSON response payloads.

Observation: Adapting the Python BDD framework for API testing with a data-driven approach significantly simplifies test coverage expansion. Passing variables through Gherkin steps allows effortless addition of new test datasets without changing underlying Python logic.

Conclusion: The data-driven API automation framework was successfully created and executed in Python Behave, meeting all functional criteria for Milestone 3 Assignment 2.